AirMUA: Dynamically Allocating Users on the Fly in Collaborative Large-Small Model Systems

Guangming Cui, Xiaolong Xu, *Senior Member, IEEE*, Yiwen Zhang, Lianyong Qi, *Senior Member, IEEE*, and Zhipeng Cai, *Fellow, IEEE*

***Abstract*—In a collaborative large-small language model (CLSLM) system, application providers can deploy their small-scale intelligent services at the edge to serve the mobile users. Due to the personalized characteristics of edge small models, an application provider must properly accommodate its users for service and/or data provision. In general, a cost-effective mobile user allocation (MUA) solution must serve the maximum number of users with minimum edge resources by considering mobile users' preferences. However, existing studies of MUA assume that users' and edge servers' available resources are static and neglect the matching between mobile users' preferences and the edge small models' personalization. This is unrealistic in real-world CLSLM systems where the mobility of users changes dynamically and the edge servers' available resources differ periodically. Therefore, users must be allocated on the fly to edge servers whose available resources also vary over time. Most existing MUA approaches are impractical in real-world CLSLM systems because they are usually designed to address the MUA problem offline, and their performance is usually poor. In this paper, AirMUA, a novel online scheme, is designed, aiming to solve the dynamic MUA (DMUA) problem. AirMUA allocates users to edge servers as they join the system and utilizes system resources dynamically released by user departures. Comprehensive experiments and analyses are conducted to evaluate its performance systematically. Extensive experimental results show the high performance of AirMUA, which ensures its practicality in real-world CLSLM systems.**

Received 2 April 2025; revised 15 September 2025; accepted 11 November 2025. Date of publication 14 November 2025; date of current version 6 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62502220, in part by the Natural Science Foundation of Jiangsu Province under Grant BK20240692, in part by the Natural Science Foundation of the Jiangsu Higher Education Institutions of China under Grant 24KJB520021, in part by the Fundamental Research Funds for the Central Universities, JLU under Grant 93K172025K04, and in part by the Startup Foundation for Introducing Talent of NUIST. Recommended for acceptance by X. Chen. *(Corresponding author: Xiaolong Xu)*

Guangming Cui is with the School of Software, Nanjing University of Information Science & Technology, Nanjing 211544, China, also with the Jiangsu Province Engineering Research Center of Advanced Computing and Intelligent Services, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210093, China, and also with the Laboratory of Symbolic Computation and Knowledge Engineering of Ministry of Education, Jilin University, Changchun 130012, China (e-mail: gcui@nuist.edu.cn).

Xiaolong Xu is with the School of Software, Nanjing University of Information Science and Technology, Nanjing 211544, China, and also with the Jiangsu Province Engineering Research Center of Advanced Computing and Intelligent Services, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210093, China (e-mail: xlxu@ieee.org).

Yiwen Zhang is with the School of Computer Science and Technology, Anhui University, Anhui 230031, China (e-mail: zhangyiwen@ahu.edu.cn).

Lianyong Qi is with the College of Computer Science and Technology, China University of Petroleum (East China), Qingdao 257099, China (e-mail: lianyongqi@gmail.com).

Zhipeng Cai is with the Department of Computer Science, Georgia State University, Atlanta, GA 30302 USA (e-mail: zcai@gsu.edu).

Digital Object Identifier 10.1109/TMC.2025.3632801

***Index Terms*—Dynamic mobile user allocation, collaborative large-small language model system, mobile edge computing.**

I. Introduction

RECENTLY, the world has experienced an unprecedented surge in the proliferation of mobile devices, including smartphones, tablets, and wearable gadgets, which has fundamentally transformed how individuals interact with digital services. This explosion in device numbers, however, is accompanied by inherent limitations in computational resources due to physical size constraints [1], [2]. Unlike desktop computers or data center servers, mobile devices often lack the processing power, memory, and energy efficiency required to execute complex applications such as real-time video analytics or large-scale machine learning models [3], [4]. These computational bottlenecks have necessitated innovative solutions to bridge the gap between user demands and device capabilities [5], [6].

Cloud computing emerged as an early remedy, offering remote processing capabilities through centralized data centers. This paradigm shift allowed mobile users to offload intensive tasks to powerful cloud servers, thereby extending the functional boundaries of their devices [7], [8]. However, the centralized nature of cloud infrastructure introduced critical latency challenges, particularly for applications requiring real-time responsiveness [9], [10]. For instance, latency-sensitive services like autonomous driving, remote surgery, or immersive VR/AR experiences demand sub-50 ms end-to-end delays [11], which traditional cloud architectures struggle to consistently achieve due to geographical distances and network congestion [12], [13].

To handle these limitations, mobile edge computing (MEC) was conceptualized as a decentralized extension of cloud computing. By deploying edge servers at the network's periphery - in cellular base stations, Wi-Fi access points, or local data centers - MEC significantly reduces the physical distance between service providers and mobile users. This proximity enables ultra-low latency processing, enhanced bandwidth utilization, and improved energy efficiency by keeping data localized [11], [14], [15]. The distributed architecture of MEC introduces new operational complexities, particularly in managing user allocation across geographically dispersed edge nodes [15], [16], [17].

Especially, in the collaborative large-small language model (CLSLM) system, the large language model-based services are deployed in the cloud, and the small model-based services are usually deployed at the network edge. Then, the large models and small models can collaborate to provide personalized service to

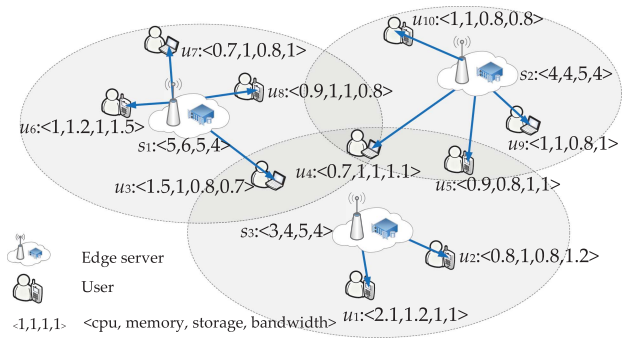


Fig. 1. Motivation Example.

their mobile users based on the users' preferences [18]. However, the Mobile User Allocation (MUA) problem in CLSLM environments presents significant challenges due to three primary factors: (1) Dynamic User Mobility, where users frequently change locations and service requirements [10], [19]; (2) Heterogeneous Edge Resources, as edge servers vary in computational capacity, storage, and network connectivity [20]; and (3) Preference-Model Alignments that mandate the two core components of alignment between user preferences and model's resources personalization. Traditional optimization approaches often adopt static, offline models that assume fixed user distributions and resource availability [21], [22]. These methods, while effective in controlled environments, fail to adapt to real-world dynamics where user arrivals/departures and edge server workloads fluctuate unpredictably.

Fig. 1 depicts an illustrative example of a CLSLM system comprising three edge servers, which have been deployed with the small-model services s_1, s_2, and s_3. At the moment, ten users, $u_1, \ldots, u_{10}$, are waiting to access those services. Edge servers' available resources are denoted by $\langle$CPU, memory, storage, bandwidth$\rangle$. When allocating these users, the *coverage constraint* must be fulfilled. For example, u_6 and u_8 can only be allocated to s_1 while u_3 can be allocated to either s_1 or s_3. The *capacity constraint* must also be fulfilled. An edge server's available resources must be sufficient to meet the overall resource demands of the users allocated to the edge server. For example, u_3, u_4, u_6, u_7, and u_8 can not be served by s_1 simultaneously because s_1's available resources are unable to meet their resource demands, i.e., $\langle 4.8, 5.2, 4.6, 5.1\rangle$. As proven in [15], [23], the optimal solution in a one-time slot cannot be found in a reasonable time since it is NP-hard. The key is to allocate users who have multiple choices properly.

An edge server with limited resources may be unable to serve other users if a user is assigned to it. Take Fig. 1 as an example. If u_3 is allocated to s_3, u_1 cannot be served by any edge servers. Furthermore, Fig. 1 serves as an exemplary illustration of the MUA problem within a single time slot. However, for different time slots, users might arrive and depart the system randomly, making it impossible to predict their arrivals and departures accurately. In this case, the dynamic MUA problem should be considered. Offline approaches, like those proposed in [11], [16], [19], will suffer from poor performance without considering system dynamics. Additionally, in the CLSLM system, the matching between user preferences and heterogeneous edge model resources intensifies the mobile user allocation problem. Thus, in this paper, our user allocation solution for edge small models directly enables this collaboration by ensuring edge nodes have sufficient compute/memory bandwidth to: (a) run small models efficiently for routine tasks by avoiding unnecessary offloading to the cloud, which reduces latency and cloud cost, and (b) process offloading triggers and transmit offloaded tasks to the cloud to avoid bottlenecks that break CLSLM workflow. Without optimized edge resources, the CLSLM system either (i) underuses edge capacity, resulting in forcing excessive offloading, straining cloud resources, or (ii) overcommits edge resources, causing small model latency and missing real-time triggers for offloading.

In the context of the Dynamic mobile user allocation (DMUA) problem, during each time slot t, each user u_i arrives in the system at a time slot f_i^a and will stay for several time slots f_i^s dynamically. When allocating a user u_i to an edge server s_j, we have knowledge of the states of all users in the previous time slots t_1 to $t-1$. However, we lack knowledge about future users arriving in the subsequent time slot $t+1$. In this setting, the DMUA problem tries to serve the maximum number of users to the minimum number of edge servers by considering the matching between the personalization of small models and the preferences of mobile users in the current time slot, and leaves the maximum available resources to allocate to the upcoming users.

Contributions: The paper proposes a theoretically guaranteed online approach to dealing with the dynamic mobile user allocation (DMUA) problem. The major contributions are as follows:

- We provide a formal formulation of the DMUA problem that takes into account all of the unique characteristics of CLSLM systems.
- To solve the DMUA problem, we design AirMUA and analyze its performance theoretically, including the time complexity and competitive ratio.
- We evaluate the performance of AirMUA using a public dataset and compare it against four representative approaches.

Section II states the DMUA problem formally. Section III presents AirMUA in a systematic manner and analyzes the competitive ratio theoretically. Section IV experimentally evaluates AirMUA's performance. Sections V and VI summarize the related work and conclusions.

II. Problem Statement

This section first introduces the profit and cost models built for DMUA. Then, we formulate the DMUA problem, present its optimization model, and prove its hardness.

A. Profit and Cost Models

For the DMUA problem, it usually includes n mobile users $U = \{u_1, \ldots, u_n\}$ with dynamic preferences, and m edge servers $S = \{s_1, \ldots, s_m\}$ which have been cached various personalized small models. Each user $u_i \in U$ joins

TABLE I
KEY NOTATIONS

Notation	Description
x_i^t	allocation decision for user u_i^t in t
$\mathbf{x}^t = (x_1^t, ..., x_n^t)$	MUA strategy in t
$\mathcal{A}$	MUA strategies over T
d	resource dimensions
$I_{\{condition\}}$	an indicator
m, n	number of edge servers and users respectively
$s_j \in S$	edge server $s_j, j \in \{1, ..., m\}$
t	any time slot
T	set of time slots
$u_i^t \in U^t$	user $u_i^t, i \in \{1, ..., n\}$
$\tau_j^{t,k}$	kth available resource of s_j in t
$\Gamma(\mathbf{x}^t)$	allocation profit produced by $\mathbf{x}^t$
$\Theta(\mathbf{x}^t)$	allocation cost incurred by $\mathbf{x}^t$
$\tau_j^t = (\tau_j^{t,k})$	available resource of s_j in time slot t
$\kappa_i^t = (\kappa_i^{t,k})$	u_i^t's resource demand
$\xi_j^k(\mathbf{x}^t)$	s_j's kth used resource
$\xi_j(\mathbf{x}^t)$	s_j's used resource
$\mu_j^k(\mathbf{x}^t)$	s_j's kth unused resource
$\mu_j(\mathbf{x}^t)$	s_j's unused resource
λ^k	application provider's preference of kth-dimensional resource
γ	application provider's priority parameter of allocation cost
ς	set of used edge servers

the system at a time slot $f_i^a \in T$ and departs at a time slot f_i^d with the resource demands $\kappa_1 = \{\kappa_i^1, \ldots, \kappa_i^d\}, d \in \{\text{cpu, memory, storage, etc.,}\}$. Each edge server $s_j \in S$ has been equipped with a set of available resources $\tau_j = \{\tau_j^1, \ldots, \tau_j^d\}$. Then, based on the above settings, to ease the description, the users in t can be denoted as $U^t = \{u_1^t, \ldots, u_n^t\}$ with a variable-sized resource demands $\kappa^t = \{\kappa_1^t, \ldots, \kappa_n^t\}$, where $\kappa_i^t = \{\kappa_i^{t,1}, \ldots, \kappa_i^{t,d}\}$ denotes the multidimensional resources required to serve user u_i^t, for instance, CPU, memory, storage, etc., and $u_i^t \in U \wedge f_i^a \leq t \leq f_i^d$. Similarly, in time slot t, the available resource of $S = \{s_1, \ldots, s_m\}$ can be denoted as $\tau^t = \{\tau_1^t, \ldots, \tau_m^t\}$, where $\tau_j^t = \{\tau_j^{t,1}, \ldots, \tau_j^{t,d}\}$. Then, the t-DMUA problem tries to accommodate the maximum number of users at the minimum cost, and the maximum number of unused available resources is left to serve the upcoming users in the future. Table I summarizes the notations used throughout.

First, let $N(u_i^t)$ denote the set of user u_i^t's nearby edge servers, namely *nearby edge servers*,

$$N(u_i^t) = \{s_j | u_i^t \in coverage_j\}, \tag{1}$$

where $coverage_j$ represents s_j's geographical coverage, and $s_j \in S$.

Next, according to (1), the allocation decision for u_i^t in each time slot t is denoted by x_i^t, where $x_i^t \in \{0\} \bigcup \{j | s_j \in N(u_i^t)\}$. Here $x_i^t = 0$ means that u_i^t fails to be served in time slot t, $x_i^t = j$ denotes that u_i^t is served by s_j in t. All the MUA strategy in t is represented by $\mathbf{x}^t = (x_1^t, \ldots, x_n^t)$.

Allocation Profit: The profit in a DMUA scenario is produced by serving its users. Thus, the application provider first tries to maximize the number of users allocated. In t, the allocation profit produced by an MUA strategy $\mathbf{x}^t$ is defined as:

$$\Gamma(\mathbf{x}^t) = \sum_{u_i^t \in U^t} I_{\{x_i^t \neq 0\}}, \tag{2}$$

where $I_{\{condition\}}$ is an indicator that equals to 1 when *condition* is true. Otherwise, 0.

The allocation profit produced by all the MUA strategies over T $\mathbf{x}$ is defined as:

$$\Gamma(\mathbf{x}) = \sum_{t \in T} \Gamma(\mathbf{x}^t). \tag{3}$$

Allocation Cost: From the application provider's perspective, deploying its application instances on more edge servers incurs more costs [15], [16], [24]. Thus, the number of servers used should be minimized, i.e., the second objective of the DMUA problem. Given an allocation decision x_i^t ($x_i^t \in \mathbf{x}^t$) of u_i^t in time slot t, the cost incurred by allocating u_i^t, $u_i^t \in U^t$, is calculated by,

$$\Theta(\mathbf{x}^t) = \sum_{s_j \in S} I_{\left\{\sum_{u_i^t \in U^t} I_{\{x_i^t = j\}} \neq 0\right\}}. \tag{4}$$

Then, the allocation cost by a series of MUA strategies formulated over time T, denoted by $\mathbf{x}$, is defined as,

$$\Theta(\mathbf{x}) = \sum_{t \in T} \Theta(\mathbf{x}^t). \tag{5}$$

Similar to many studies on the MUA problem [11], [15], [23], the cost-effective DMUA attempts to maximize the **Allocation Profit** and minimize the **Allocation Cost** over time while fulfilling the coverage and capacity constraints. Based on the allocation profit and cost models, the DMUA problem first aims to maximize the allocation profit, i.e., (6a), then if there are multiple solutions, the one with the minimum allocation cost measured by (6b) will be selected as the final solution:

$$\max \Gamma(\mathbf{x}) = \sum_{t \in T} \Gamma(\mathbf{x}^t), \tag{6a}$$

$$\min \Theta(\mathbf{x}) = \sum_{t \in T} \Theta(\mathbf{x}^t). \tag{6b}$$

s.t.

$$x_i(t) \in \{0\} \bigcup \{j | s_j \in N(u_i^t)\}, \forall u_i^t \in U^t, s_j \in S, t \in T, \tag{7a}$$

$$\sum_{u_i^t \in U^t : x_i(t)=j} \kappa_i^k \le \tau_j^k,\ \forall k \le d, s_j \in S, t \in T. \tag{7b}$$

Objective (6) aims to maximize the total allocation profit produced and minimize the total allocation cost incurred by the MUA strategies over T. Constraint (7a) enforces the coverage constraint in each time slot $t \in T$, as described in Section I. Similarly, Constraint (7b) enforces the capacity constraint in each time slot $t \in T$.

III. AirMUA

Due to the resource constraint and the coverage constraint, the available resources on an edge server for accommodating users may vary drastically across different time slots. The available resources in the current time slot may be exhausted by other application providers in the next time slot. Thus, from the application provider's perspective, the best practice is often to leverage the available resources in the current time slot to serve as many users as possible. This is different from many online algorithms, e.g., those aiming to balance or maximize resource utilization over time. In this section, we present the design of AirMUA in detail and analyze its approximation ratio theoretically.

A. Algorithm Design

As detailed in Section II, the optimal solution for the DMUA problem can only be obtained when comprehensive information about mobile users and edge servers across all time slots is available, encompassing users' locations, preference resource requirements, and personalized edge servers' capacities. However, this is impractical for DMUA, as in any given time slot $t \in T$, information about users and edge servers in subsequent time slots after t remains unknown. Consequently, in real-world DMUA scenarios, users must be allocated to edge servers dynamically and in real-time as they arrive in the CLSLM system.

Given the set of time slots T, the users U^T across T, and the edge servers S in the CLSLM system, AirMUA, our online DMUA approach, solves the corresponding t-DMUA problem, i.e., the MUA problem in the time slot t, to accommodate users when these users come in each time slot without any knowledge about future users or server states. Due to the proven NP-hardness of the t-DMUA problem, AirMUA employs an approximate approach that prioritizes allocating users demanding the most resources to their neighboring edge servers based on available resources. To match the personalization of small models and the various preferences of mobile users, inspired by Dominant Resource Fairness [25], [26], we proposed the concept of Demand Dominance Coefficient, defined as follows:

Definition 1 (Demand Dominance Coefficient): Given a user u_i^t and its resource needed with multi-dimensional requirements $\kappa_i^t = \{\kappa_i^{t,1}, \ldots, \kappa_i^{t,d}\}$, to match the user preference κ_i^t and the model resource personalization τ_j^t, u_i^t's demand dominance coefficient, denoted by $\kappa_i^{t,*}$, is the maximum ratio of u_i^t's resource demand $\kappa_i^{t,k} \in \kappa$ over the kth-dimensional resource available on $s_j \in N(u_i^t)$, i.e.,

$$\kappa_i^{t,*} \triangleq \max_{k \le d} \frac{\kappa_i^{t,k}}{\max_{s_j \in N(u_i^t)} \tau_j^{t,k}}. \tag{8}$$

Now, for all users in U^t, they are sorted in non-decreasing order by their demand dominance coefficients $\kappa_i^{t,*}$ of u_i^t, ($\kappa_1^{t,*} \le \kappa_2^{t,*} \le \ldots \le \kappa_n^{t,*}$).

Please recall that $\mathcal{U}_j(\mathbf{x}^t)$ represents all the accommodated users of edge server s_j. To facilitate the discussion of AirMUA next, we define two key terms: 1) $\xi_j(\mathbf{x}^t)$ is used to denote s_j's used resources to accommodate $\mathcal{U}_j(\mathbf{x}^t)$; and 2) $\mu_j(\mathbf{x}^t)$ is used to denote s_j's unused resources based on MUA strategy $\mathbf{x}^t$. The mathematical expressions are as follows.

$$\xi_j^k(\mathbf{x}^t) \triangleq \sum_{u_i^t \in \mathcal{U}_j(\mathbf{x}^t)} \kappa_i^{t,k}, \tag{9}$$

$$\xi_j(\mathbf{x}^t) \triangleq \sum_{k \le d} \lambda^k \cdot \xi_j^k(\mathbf{x}^t), \tag{10}$$

$$\mu_j^k(\mathbf{x}^t) \triangleq \tau_j^{t,k} - \xi_j^k(\mathbf{x}^t), \tag{11}$$

$$\mu_j(\mathbf{x}^t) \triangleq \sum_{k \le d} \lambda^k \cdot \mu_j^k(\mathbf{x}^t), \tag{12}$$

where $\lambda^k (k \le d)$ is the application provider's preference for the kth-dimensional resource.

For the DMUA problem, given n mobile users U with preferences across T, and m edge servers S with deployment of various personalized small models, Algorithm 1 is first used to formulate the allocation strategy $\mathbf{x}$ of U across T of AirMUA. First, take the resource demands and the arrival and departure time of U, and the available resources of S as input, Algorithm 1 first initializes three global parameters, i.e., $\mathbf{x}$, τ^t, and U^t. Then, for each time slot $t \in T$, the pre-processes of the DEAU system are provided, including the removal of the used resources of the departing users, and the inclusion of the new arrival users. Next, the new system state (U^t and τ^t) is input to Algorithm 2 to formulate the allocation strategy of the t-DMUA problem.

Given the set of mobile users U and edge servers S in a specific time slot t, AirMUA utilizes Algorithm 2 to determine the MUA strategy for that time slot, denoted as $\mathbf{x}^t$. Here's a detailed breakdown of the algorithm's steps:

1) *Initialization:* The algorithm begins by initializing two global parameters: $\mathbf{x}^t$ (which will eventually represent the MUA strategy for time slot t) and ς (a set to track which edge servers have already been considered in the allocation process) (Lines 1-4).
2) *Sorting Users:* The users in U^t are sorted in non-decreasing order based on their demand dominance coefficient. This coefficient likely reflects the relative importance or resource requirements of each user, aiding in efficient resource allocation.
3) *Iterative Allocation:* The algorithm then enters an iterative process to formulate $\mathbf{x}^t$ by allocating users to edge servers (Lines 6-23).
4) *Server Initialization:* For each edge server s_j, that has not yet been considered (i.e., $s_j \in S - \varsigma$), the algorithm

Algorithm 1: Formulate DMUA Over T.

Input: a set of mobile users $U = \{u_1, \ldots, u_N\}$ with the preferred resource demands $\kappa = \{\kappa_1, \ldots, \kappa_N\}$ and the set of arrival time $f^a = \{f_1^a, \ldots, f_N^a\}$ over T, a set of departure time $f^d = \{f_1^d, \ldots, f_N^d\}$ across T, a set of edge servers $S = \{s_1, \ldots, s_m\}$ with the available resources $\tau = \{\tau_1, \ldots, \tau_m\}$.
Output: allocation strategy $\mathbf{x} = \{x_1, \ldots, x_N\}$ of users U

1: **initialization**
2: DMUA strategy $\mathbf{x} \leftarrow (0, \ldots, 0)$ where $\mathbf{x} = \{x_1, \ldots, x_N\}$
3: t-DMUA strategy $\mathbf{x}^t \leftarrow (0, \ldots, 0)$ where $\mathbf{x}^t = \{x_1^t, \ldots, x_n^t\}$
4: a set of active mobile users in time slot t: $U^t = \{\}$
5: the available resources of all edge servers S in time slot t: $\tau^t = \{\}$
6: set of used edge server: $\varsigma \leftarrow \emptyset$
7: **end initialization**
8: **for all** $t \in T$ **do**
9: //Remove the used resources of the departure users
10: **if** t==1 **then**
11: update the current available resources in time slot t: $\tau^t = \tau$
12: **for all** $u_i \in U$ **do**
13: **if** $f_i^a \le t \le f_i^d$ **then**
14: update $U^t = U^t \bigcup \{u_i\}$
15: **end if**
16: **end for**
17: **else**
18: **for all** $u_i^{t-1} \in U^{t-1}$ **do**
19: **if** $f_i^d > t$ **then**
20: remove the used resources of u_i
21: **end if**
22: **end for**
23: **end if**
24: // Include the new arrival users
25: **for all** $u_i \in U$ **do**
26: **if** $f_i^a \le t \le f_i^d$ **then**
27: update $U^t = U^t \bigcup \{u_i\}$
28: **end if**
29: **end for**
30: //Formulate the t-DMUA strategy
31: Formulate the MUA strategy in time slot t with the input U^t and τ^t
32: **end for**
33: **return** $\mathbf{x}$

Algorithm 2: Formulate the MUA Strategy in Time Slot t.

Input: a set of mobile users $U^t = \{u_1^t, \ldots, u_n^t\}$ with the preferred resource demands $\kappa^t = \{\kappa_1^t, \ldots, \kappa_n^t\}$, a set of edge servers $S = \{s_1, \ldots, s_m\}$ with the personalized available resources $\tau^t = \{\tau_1^t, \ldots, \tau_m^t\}$.
Output: allocation strategy $\mathbf{x}^t = \{x_1^t, \ldots, x_n^t\}$ of users U^t

1: **initialization**
2: MUA strategy in time slot t: $\mathbf{x}^t \leftarrow (0, \ldots, 0)$ where $\mathbf{x}^t = \{x_1^t, \ldots, x_n^t\}$
3: set of used edge server: $\varsigma \leftarrow \emptyset$
4: **end initialization**
5: sort users in U^t in non-decreasing order based on (8)
6: **while** $S - \varsigma \neq \emptyset$ **or** more users could be allocated **do**
7: **for all** $s_j \in S$ **do**
8: initialize the u_i^t's allocation profile $\mathbf{x}_j^t \leftarrow (0, \ldots, 0)$ where $\mathbf{x}_j^t = \{x_{1,j}^t, \ldots, x_{n,j}^t\}$
9: update the demand dominance coefficient of each unallocated user based on (8)
10: resort unallocated users in non-decreasing order based on (8)
11: **for all** $u_i^t \in U^t$ **do**
12: **if** $x_i^t = 0$ **then**
13: **if** $s_j \in N(u_i^t)$ **and** $\sum_{u_l^t \in U^t : x_{l,j}^t = 1} \kappa_l^{t,k} + \kappa_i^{t,k} \le \tau_j^{t,k}, k \le d$ **then**
14: allocate u_i^t to s_j, i.e., $x_{i,j}^t \leftarrow 1$
15: **end if**
16: **end if**
17: **end for**
18: calculate $\mu_j(\mathbf{x}_j^t)$
19: **end for**
20: find s_{j^*}, $j^* = \arg\min \mu_j(\mathbf{x}_j^t)$, $s_j \in S - \varsigma$
21: **for all** $u_i^t \in U^t$ **and** $x_{i,j^*}^t = 1$ **do**
22: update $\mathbf{x}^t$: $x_i^t \leftarrow 1$
23: **end for**
24: update the set of used edge server: $\varsigma \leftarrow \varsigma \bigcup \{s_{j^*}\}$
25: **end while**
26: **return** $\mathbf{x}^t$

initializes a vector $\mathbf{x}_j^t$ which indicates whether each user $u_i^t \in U^t$ is allocated to s_j (Line 8).

5) *User Allocation:* The algorithm then checks each unallocated user $u_i^t \in U^t$ (i.e., those for which $x_i^t = 0$). If allocating u_i^t to s_j satisfies both the coverage constraint (ensuring the user is within the server's service area) and the capacity constraint (ensuring the server has enough resources to serve the user), u_i^t is allocated to s_j in ascending order of user indices $i = 1, \ldots, n$ (Lines 9-15).
6) *Unused Resources Calculation:* After attempting to allocate all possible users to s_j, the algorithm calculates the unused resources of s_j (Line 16).
7) *Selection of Optimal Server:* Once all edge servers have been processed in the current iteration, the algorithm identifies the edge server s_{j^*} with the least unused resources (Line 18). The corresponding allocation vector $\mathbf{x}_{j^*}^t$ is then included in $\mathbf{x}^t$ (Lines 19-21).
8) *Update Tracked Servers:* The edge server s_{j^*} is then added to the set ς (Line 22), indicating that it has been considered in the allocation process.
9) *Iteration Continuation:* The process from Line 6 to 23 is repeated until all edge servers have been processed or no more users can be allocated to any server without violating constraints.
10) *Finally, the MUA strategy $\mathbf{x}^t$ is returned:* This strategy is stored in $\mathcal{A}$ (a repository or data structure for storing MUA strategies) and implemented in time slot t.

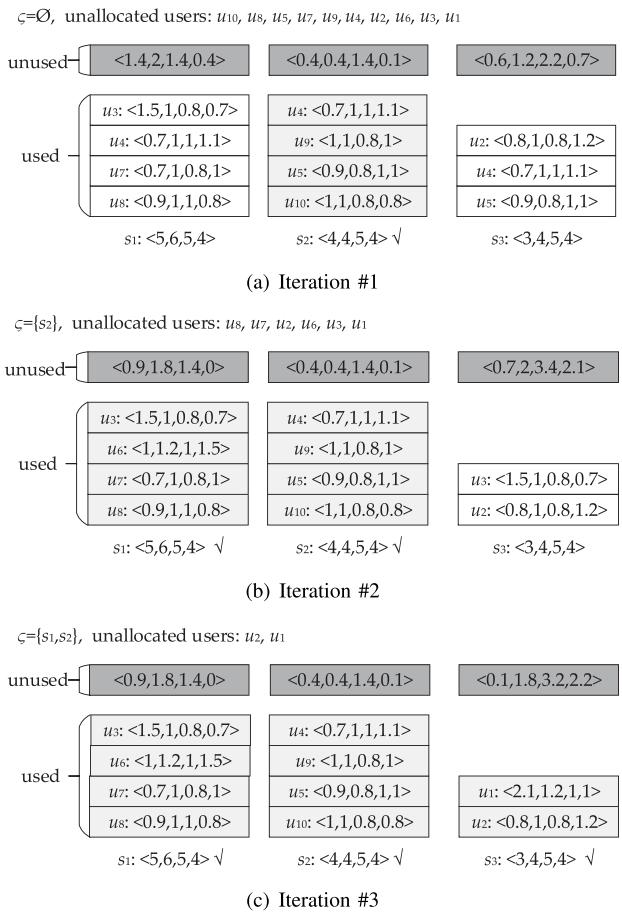


Fig. 2. An example application of AirMUA.

In $t \in T$, the maximum average number of unallocated users within the coverage of an individual edge server $s_j \in \varsigma$ is n/m. Thus, Algorithm 2 can find a feasible solution within $O(m \cdot m \cdot n/m)$ time. Thus, the maximum time overhead of AirMUA in an individual time slot is $O(mn)$.

B. Case Study

Take Fig. 1 as an example. AirMUA first calculates each user's demand dominance coefficient: $\kappa_1^{t,*} = 0.7$, $\kappa_2^{t,*} = 0.3667$, $\kappa_3^{t,*} = 0.5$, $\kappa_4^{t,*} = 0.275$, $\kappa_5^{t,*} = 0.25$, $\kappa_6^{t,*} = 0.375$, $\kappa_7^{t,*} = 0.25$, $\kappa_8^{t,*} = 0.225$, $\kappa_9^{t,*} = 0.25$, $\kappa_{10}^{t,*} = 0.2$. Then, these users are sorted in non-decreasing order by their demand dominance coefficients, i.e., $u_{10}, u_8, u_5, u_7, u_9, u_4, u_2, u_6, u_3$, and u_1. Next, Algorithm 2 will allocate these users one by one (Fig. 2).

Fig. 2(a) demonstrates iteration #1, where the set of edge servers used by $\mathbf{x}^t$ is empty, i.e., $\varsigma = \emptyset$. Then, for each edge server $s_j \in S - \varsigma$, Algorithm 2 will allocate the users by following their order to individual edge servers without considering other edge servers. For s_1, it allocates u_8, u_7, u_4, u_6 and u_3. User u_{10}, as the first user in the order, is excluded here because it is not covered by s_1. User u_6 cannot be allocated to s_1 because after u_8, u_7, u_4 are allocated to s_1, the available resources on s_1 do not suffice to accommodate u_6. Thus, only u_8, u_7, u_4 and u_3 are allocated to s_1. Next, Algorithm 2 will do the same for s_2 and s_3, and obtain three allocation profiles, one for each of the three edge servers. It then will calculate the unused resources on each edge server, i.e., $\mu_1^k(\mathbf{x}^t) = (1.4, 2, 1.4, 0.4)$, $\mu_2^k(\mathbf{x}^t) = (0.4, 0.4, 1.4, 0.1)$ and $\mu_3^k(\mathbf{x}^t) = (0.6, 1.2, 2.2, 0.7)$ for $k \leq d$. To ease the exposition, we assume that the application provider has equal preferences for different resource dimensions, i.e., $\lambda^k = 0.25$ for $k \leq d$. Thus, the overall unused resources on the three edge servers are $\mu_1(\mathbf{x}^t) = 1.3$, $\mu_2(\mathbf{x}^t) = 0.575$, and $\mu_3(\mathbf{x}^t) = 1.175$. Apparently, s_2 has the lowest unused resources overall. Thus, its allocation profile will be selected in iteration #1, and the users allocated to s_2 are removed from the set of remaining users. Edge server s_2 will be included in the set of used edge servers, i.e., $\varsigma = \{s_2\}$. Fig. 2(b) illustrates iteration #2, where the remaining users to be allocated are u_8, u_7, u_2, u_6, u_3 and u_1. Following the same procedure in iteration #1, the algorithm will produce two allocation profiles for s_1 and s_3. The one for s_1 is selected to allocate users u_8, u_7, u_6 and u_3. Finally, as shown in Fig. 2(c), in iteration #3, the allocation profile for edge server s_3 will be finalized to accommodate the remaining users, including u_2 and u_1. All 10 users are now allocated, and Algorithm 2 completes.

C. Performance Analysis

To analyze the performance of AirMUA theoretically, a Lemma 1 of each edge server that has cached a personalized small model is first introduced as follows.

Lemma 1: Given an edge server s_j, if it is unable to allocate mobile user u_i^t ($s_j \in N(u_i^t)$), this implies that mobile user u_i^t's multi-dimensional resource demand in at least one of the dimensions cannot be fulfilled, say the kth dimension:

$$\kappa_i^{t,k} > \mu_j^k(\mathbf{x}^t) + \sum_{u_l^t \in \mathcal{U}(\mathbf{x}^t), l>i, x_l^t = j} \kappa_l^{t,k}. \tag{13}$$

Proof: For any $\mathbf{x}^t$, the final unused resources of s_j are $\mu_j^k(\mathbf{x}^t)$, and the used resources by the allocated users who are ordered after u_i^t, i.e.,

$$\sum_{u_l^t \in \mathcal{U}(\mathbf{x}^t), l>i, x_l^t = j} \kappa_l^{t,k}.$$

Thus, if u_i^t cannot be allocated by s_j, its at least one-dimensional resource demand cannot be fulfilled by s_j, i.e.,

$$\kappa_i^{t,k} > \mu_j^k(\mathbf{x}^t) + \sum_{u_l^t \in \mathcal{U}(\mathbf{x}^t), l>i, x_l^t = j} \kappa_l^{t,k}.$$

Thus, Lemma 1 holds.□

Based on the Lemma 1, for each s_j's unallocated u_i^t ($s_j \in N(u_i^t)$), its resource demands are greater than the unused resource of s_j since the used resources by the allocated users after u_i^t is no less than 0, i.e.,

$$\kappa_i^{t,k} > \mu_j^k(\mathbf{x}^t). \tag{14}$$

Similarly, if u_i^t cannot be allocated to s_j while can be allocated to s_j' ($s_j, s_j' \in N(u_i^t)$), the available resources on s_j' should be larger than the unused resources on s_j, i.e.,

$$\mu_j^k(\mathbf{x}^t) < \tau_{j'}^{t,k}. \tag{15}$$

According to the above inequalities, the Lemma 2 can be fulfilled.

Lemma 2: In the solution obtained via Algorithm 2, for any edge server $s_j \in \varsigma$ possessing the minimum unused resource capacity, the following inequality is established:

$$\tau_j^{t,k} < 2 \sum_{u_i^t \in \mathcal{U}_j(\mathbf{x}^t)} \kappa_i^{t,k} = 2\xi_j^k(\mathbf{x}^t).$$

Proof: If u_i^t can be allocated to s_j, its resource demands should be no greater than the used resources on s_j, i.e., $\kappa_i^{t,k} \leq \sum_{u_l^t \in \mathcal{U}_j(\mathbf{x}^t)} \kappa_l^{t,k}$. Then, given two users u_{sup}^t and u_{inf}^t [1]), where u_{sup}^t and u_{inf}^t are the first and last mobile user accommodated to s_j according to $\mathbf{x}^t$, respectively. Then, from (14), we know $\mu_j^k(\mathbf{x}^t) < \kappa_{inf+1}^{t,k}$, since u_{inf}^t is the last one that can be allocated to s_j, that is, u_{inf+1}^t may not be accommendated by s_j and will instead be assigned to an alternative edge server s_j'. Now, we try to prove $\mu_j^k(\mathbf{x}^t) < \sum_{u_i^t \in \mathcal{U}_j(\mathbf{x}^t)} \kappa_i^{t,k}$ using reduction to absurdity. If there is an edge server $s_j \in \varsigma$ such that $\mu_j^k(\mathbf{x}^t) \geq \sum_{u_i^t \in \mathcal{U}_j(\mathbf{x}^t)} \kappa_i^k$, there will be

$$\kappa_{sup}^{t,k} \leq \kappa_{sup+1}^{t,k} \leq \sum_{u_i^t \in \mathcal{U}_j(\mathbf{x}^t)} \kappa_i^{t,k} \leq \mu_j^k(\mathbf{x}^t) < \kappa_{inf+1}^{t,k},$$

which contradicts the fact that $u_{inf+1}^t \leq u_{sup+1}^t$. Thus, the following inequality holds

$$\mu_j^k(\mathbf{x}^t) < \sum_{u_i^t \in \mathcal{U}_j(\mathbf{x}^t)} \kappa_i^{t,k},$$

i.e.,

$$\tau_j^{t,k} = \mu_j^k(\mathbf{x}^t) + \sum_{u_i^t \in \mathcal{U}_j(\mathbf{x}^t)} \kappa_i^{t,k}$$

$$< 2 \sum_{u_i^t \in \mathcal{U}_j(\mathbf{x}^t)} \kappa_i^{t,k} = 2\xi_j^k(\mathbf{x}^t).$$

Lemma 2 holds. □

Based on the above lemmas, Theorem 1 holds.

Theorem 1: The aggregate resources across all edge servers utilized in the allocation $\mathbf{x}^t$, hereinafter referred to as ψ, can be upper-bounded as follows:

$$\psi \triangleq \sum_{s_j \in \varsigma} \tau_j^{t,k} \leq 2\psi^*, \tag{16}$$

where ψ^* represents the resources consumed by the optimal MUA strategy $\mathbf{x}^{t,*}$ in time slot t. Algorithm 2 constitutes an approximation algorithm with a proven upper bound of 2, mathematically expressed as, $\psi/\psi^* = 2$.

Proof: From Lemma 2, there is

$$\tau_j^{t,k} < 2\xi_j^k(\mathbf{x}^t). \tag{17}$$

[1]It is important to note that all users are arranged in ascending order of their demand dominance coefficients $\kappa_i^{t,*}$, with the superscript $*$ omitted in this section for brevity.

Combining (17) and (16), there is:

$$\psi = \sum_{s_j \in \varsigma} \tau_j^{t,k} < 2 \sum_{s_j \in \varsigma} \xi_j^k(\mathbf{x}^t). \tag{18}$$

Since at most n users are allocated in the system, i.e., all the users in U, the total resources on all the edge servers used by $\mathbf{x}^t$ follow:

$$\sum_{s_j \in \varsigma} \xi_j^k(\mathbf{x}^t) \leq \sum_{u_i^t \in U^t} \kappa_i^{t,k} = \psi^*. \tag{19}$$

Thus, based on (18) and (19), the following inequality holds.

$$\psi = \sum_{s_j \in \varsigma} \tau_j^{t,k} < 2 \sum_{u_i^t \in U^t} \kappa_i^{t,k} = 2\psi^*.$$

Thus, the tight upper bound of Algorithm 2 is $\frac{\psi}{\psi^*} = 2$, which confirms that Theorem 1 holds. □

Under the assumption of uniform capacities across all edge servers, we demonstrate the competitive ratio of AirMUA via Theorem 2. It is important to note that more realistic scenarios, such as varying capacities for individual edge servers, are subject to evaluation in Section IV.

Theorem 2: The competitive ratio of AirMUA is established as 2, which is to say,

$$\frac{\Theta}{\Theta^*} \leq 2, \tag{20}$$

where Θ and Θ^* are the allocation cost incurred by the allocation strategy achieved by AirMUA and the offline optimal allocation strategy over T, respectively.

Proof: Given the assumption of uniform capacities across all edge servers in this analysis, the total resources utilized by the allocation $\mathbf{x}^t$ consequently scale linearly with the number of edge servers employed. Building upon Theorem 1, the upper bound of the allocation cost $\Theta(\mathbf{x}^t)$ is derived as follows,

$$\Theta(\mathbf{x}^t) \leq 2\Theta^*(\mathbf{x}^{t,*}), \tag{21}$$

where $\mathbf{x}^{t,*}$ is the optimal allocation strategy in time slot t, and $\Theta^*(\mathbf{x}^{t,*})$ is the allocation cost incurred by $\mathbf{x}^{t,*}$.

Then, When AirMUA terminates for all time slots T, the allocation cost incurred by $\mathcal{A} = \{\mathbf{x}^1, \ldots, \mathbf{x}^t, \ldots, \mathbf{x}^T\}$, denoted by Θ, can be calculated by,

$$\Theta = \sum_{t \in T} \Theta(\mathbf{x}^t). \tag{22}$$

Next, let Θ^* denote the allocation cost resulting from the offline optimal DMUA strategy across the time horizon T

$$\Theta^* = \sum_{t \in T} \Theta^*(\mathbf{x}^{t,*}). \tag{23}$$

Then, based on (21), (22) and (23), the following inequality holds,

$$\Theta = \sum_{t \in T} \Theta(\mathbf{x}^t) \leq 2 \sum_{t \in T} \Theta^*(\mathbf{x}^{t,*}) = 2\Theta^*. \tag{24}$$

Thus, Theorem 2 holds. □

TABLE II
EXPERIMENTAL SETTINGS

		Number of Mobile Users (n)	Number of Edge Servers (m)	Available Resources (τ)
Set #1	Set #1.1	$2, 3, 4, 5, 6, 7, 8, 9$	5	5
	Set #1.2	5	$2, 3, 4, 5, 6, 7, 8, 9$	5
	Set #1.3	5	5	$2, 3, 4, 5, 6, 7, 8, 9$
Set #2	Set #2.1	$100, 200, 300, 400, 500, 600, 700, 800$	60	5
	Set #2.2	400	$30, 40, 50, 60, 70, 80, 90, 100$	5
	Set #2.3	400	60	$2, 3, 4, 5, 6, 7, 8, 9$

TABLE III
AVERAGE EXPERIMENTAL RESULTS

		Optimal	AirMUA	OL-MUA	MUAGame	MCF	Random
Set #1.1	Allocation Profit	5.5	5.4275	5.4215	5.39	5.35125	5.33
	Allocation Cost	1.986	2.3162	2.5013	2.7475	2.8967	3.7975
	Time Overhead(ms)	4448.3	0.0062	0.0081	0.0134	0.0025	0.0075
Set #1.2	Allocation Profit	5.0	4.95125	4 9013	4.7191	4.6491	4.4546
	Allocation Cost	2.2383	2.3396	2.4071	2.5195	2.6363	4.1149
	Time Overhead(ms)	372.14	0.0287	0.0379	0.0412	0.0062	0.0037
Set #1.3	Allocation Profit	4.7775	4.685	4.68	4.6674	4.6575	4.395
	Allocation Cost	2.0651	2.3125	2.49	2.7125	2.8025	3.7649
	Time Overhead(ms)	69.455	0.0125	0.0327	0.0231	0.0001	0.0001
Set #2.1	Allocation Profit	-	181.7783	173.6028	165.5261	149.5768	126.4185
	Allocation Cost	-	52.2843	55.1028	57.3732	60.7396	63.5848
	Time Overhead(ms)	-	2.2376	2.3749	25.4411	0.2291	0.0335
Set #2.2	Allocation Profit	-	202.6121	190.3846	169.7134	152.0576	137.2363
	Allocation Cost	-	57.7719	59.8969	63.0301	64.5228	68.2617
	Time Overhead(ms)	-	2.3399	2.9952	28.0124	0.2016	0.0373
Set #2.3	Allocation Profit	-	207.0392	190.1664	178.3578	169.2432	156.1383
	Allocation Cost	-	50.3338	53.4588	55.8139	57.1032	61.5926
	Time Overhead(ms)	-	7.5718	2.0459	25.6262	0.1741	0.0348

IV. PERFORMANCE EVALUATION

In addition to the theoretical analysis of AirMUA in Section III-C, this section presents an experimental evaluation of the performance of AirMUA experimentally on a Windows machine with Java.

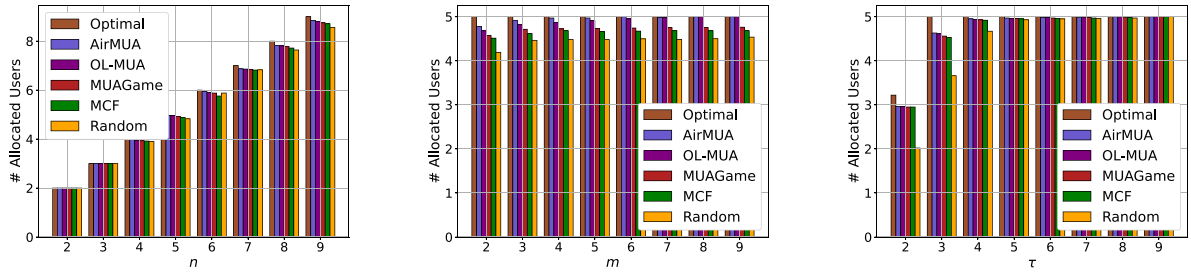
A. Experimental Setup

A public MUA dataset [23] that has been widely used in research in the field of CLSLM [11], [14], [15], [16] has been used in this paper. The dataset encompasses the geographic locations of base stations (edge servers) and users within the Melbourne CBD area. In order to assess the performance of AirMUA across diverse scenarios, the values of 1) n; 2) m; and 3) τ vary, as detailed in Table II. In the experiments, edge servers' remaining available resources vary across individual time slots, following different normal distributions, to simulate their resource heterogeneity and dynamics. Similarly, user arrivals in individual edge servers' coverage areas also vary over time, following different normal distributions.

B. Performance Metrics

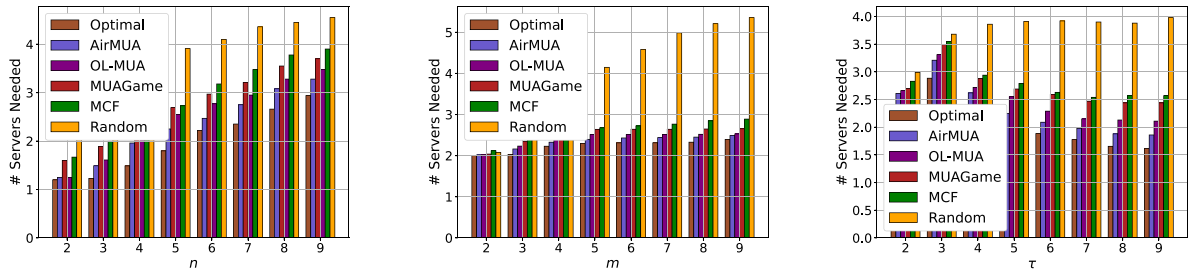
The performance of AirMUA and the competing approaches is measured by three metrics:

1) *Allocation Profit:* This metric indicates the ability of an approach to allocate the most users, the higher the better.
2) *Allocation Cost:* This metric indicates the ability of an approach to accommodate all the allocated users with the fewest edge servers, the lower the better.
3) *Time Overhead:* This metric measures the time overhead needed to find a solution. This is an important efficiency



(a) Allocation Profit vs. n (Set #1.1) (b) Allocation Profit vs. m (Set #1.2) (c) Allocation Profit vs. τ (Set #1.3)

Fig. 3. Allocation Profit (Set #1).



(a) Allocation Cost vs. n (Set #1.1) (b) Allocation Cost vs. m (Set #1.2) (c) Allocation Cost vs. τ (Set #1.3)

Fig. 4. Allocation Cost (Set #1).

indicator in CLSLM systems where low latency is always demanded.

C. Competing Approaches

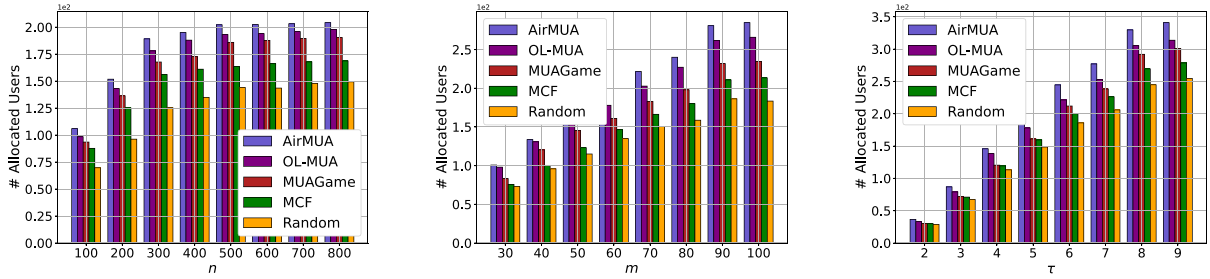
In Set #1, we evaluate AirMUA against the following four approaches. In Set #2, Optimal is excluded since the t-DMUA problem is NP-hard, so Optimal cannot find a result in a reasonable time.

1) *Optimal:* This baseline approach employs IBM's CPLEX Optimizer v12.2 to solve the model presented in Section II for finding the optimal MUA solutions in every time slot.
2) *Random:* In each time slot, this baseline approach randomly allocates users.
3) *MCF [16]:* In each time slot, this heuristic approach always attempts to allocate users with the most available resources.
4) *MUAGame [19], [27]:* In each time slot, this approach formulates the t-DMUA problem as a game and allocates users based on the Nash equilibrium.
5) *OL-MUA [28]:* For the dynamic MUA problem, this approach formulates the DMUA problem by taking into account wireless communication while neglecting the personalization of edge resources.

D. Effectiveness Evaluation

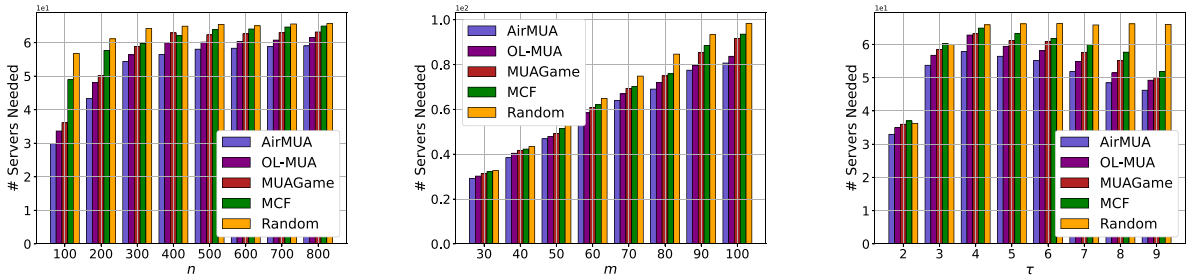
Figs. 3 and 4 illustrate the performance of the six methodologies as a function of n, m, and τ. Overall, Optimal can achieve the highest performance than other approaches. AirMUA demonstrates the second-highest performance overall. Compared to Optimal, AirMUA serves only 1.39% fewer users with 11.29% more edge servers on average.

Fig. 3(a) shows that more users usually lead to a linear increase in the allocation profit achieved by all the approaches. It can be expected when the overall resource demands of all the users are not greater than the capacity of all the edge servers in the system. Similarly, the overall allocation cost increases linearly (Fig. 4(a)). Fig. 3(b) demonstrates that the allocation cost of Optimal, AirMUA, OL-MUA, MUAGame, and MCF decreases when m increases. The background reason is that it is more likely for these approaches to allocate all the users to a few edge servers with the most available resources. It even allows AirMUA to achieve similar performance as Optimal in scenarios with $n \geq 6$ where ample resources are available on most edge servers. Without taking this advantage, Random tends to spread all users across the edge servers. Analogous to Set #1.2, the augmentation of available resources in Set #1.3 leads to an increase in allocation profit, as shown in Fig. 3(c). This is attributable to the fact that additional available resources can generally accommodate more users, with the exception of scenarios where all users within their coverage have already been allocated, such as when $\tau \geq 4$ for Optimal and AirMUA. Accordingly, allocation cost achieved by Optimal, AirMUA, OL-MUA, MUAGame and MCF experience an increase and then a decrease (Fig. 4(c)). Similar to Set #1.2, Random fails to capitalize on the increase in edge servers' resources and thus needs more edge servers when τ increases.



(a) Allocation Profit vs. n (Set #2.1) (b) Allocation Profit vs. m (Set #2.2) (c) Allocation Profit vs. τ (Set #2.3)

Fig. 5. Allocation Profit (Set #2).



(a) Allocation Cost vs. n (Set #2.1) (b) Allocation Cost vs. m (Set #2.2) (c) Allocation Cost vs. τ (Set #2.3)

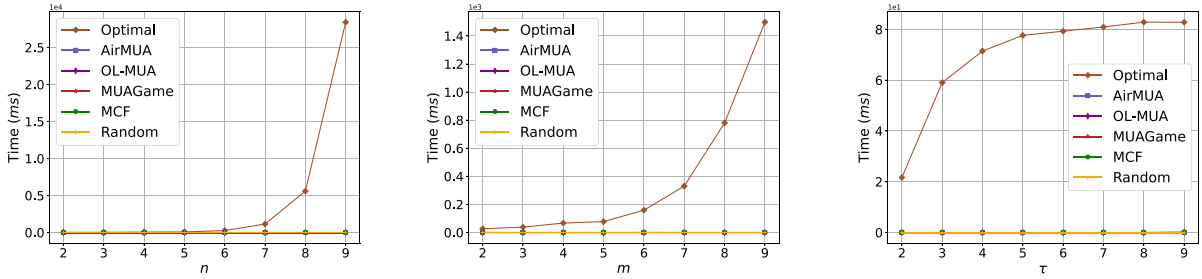
Fig. 6. Allocation Cost (Set #2).

Figs. 5 and 6 show the performance of the five approaches in experiment Sets #2.1 - #2.3 and demonstrate the impacts of the three parameters. On average across Sets #2.1 - #2.3, AirMUA achieves 20.79%, 25.63%, and 40.80% higher allocation profit than MUAGame, MCF, and Random, respectively. To accommodate these users, a lower allocation cost is obtained, specifically, 11.49% fewer than MUAGame, 10.31% than MCF, and 16.84% than Random.

In Set #2.1, AirMUA achieves 20.12%, 21.37% and 44.95% higher allocation profit than OL-MUA, MUAGame, MCF, and Random, respectively (Fig. 5(a)). Meanwhile, it outperforms MUAGame, MCF, and Random in minimizing allocation cost, by 11.24%, 9.07%, and 18.34%, respectively, as depicted in Fig. 6(a). As n increases, both allocation profit and allocation cost first increase rapidly and then stabilize. When n increases from 100, edge servers retain the capacity to serve all users. However, as n continues to rise, the available resources are fully used and can only accommodate some of the users in the system. It is also the same reason why allocation cost first increases and then stabilizes for all the approaches, as shown in Fig. 6(a). Set #2.2 shows that AirMUA outperforms OL-MUA, MUAGame, MCF, and Random by 15.35%, 21.15%, 33.22% and 46.13%, respectively, in the allocation profit as shown in Fig. 5(b), and by 10.21%, 10.33%, 10.05% and 14.50%, respectively, in allocation cost (Fig. 6(b)). When m varies, there will be more edge servers that can provide more resources in total to accommodate users. AirMUA can properly leverage the most powerful ones among all the edge servers to serve more users. In Set #2.3, from Figs. 5(c) and 6(c), AirMUA again achieves the highest Allocation profit with the minimum allocation cost. The increase in τ increases the average capacity of each individual edge server for accommodating users. In this case, the four approaches can accommodate more users on fewer edge servers for the same reason as in Set #2.2. Thus, the trends indicated in Fig. 5(c) are similarly demonstrated in Fig. 5(b). In the meantime, when τ increases, individual edge servers can always serve more users. This difference between Set #2.2 and Set #2.3 shows that the decrease in allocation cost does not slow down (Fig. 6(c)).

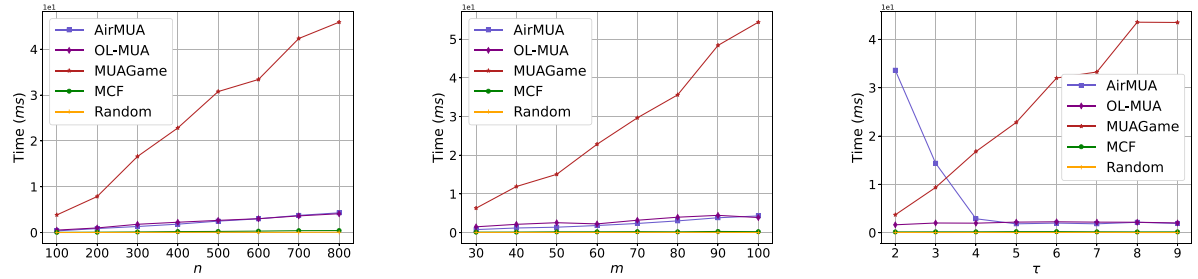
E. Efficiency Evaluation

Figs. 7 and 8 demonstrate the time overhead of different approaches within Set #1 and Set #2. As depicted in Fig. 7(a) and (b), the time overhead of Optimal grows exponentially when n and m increases. In comparison to Optimal, the time consumption of the remaining four approaches is negligible. Evidently, Optimal is not able to tackle the DMUA problems effectively, especially the larger-scale DMUA problems. This corroborates the NP-hardness of the MUA problem, which was analyzed in [14], [15], [23]. In Set #1.3, as τ increases from 2 to 9, additional resources are provisioned, enabling individual edge servers to serve more users. In this way, individual users have more options on average to be allocated. As a result, Fig. 7(c) shows that Optimal requires increased computational time to determine a solution. As τ continues to increase, a greater number of users can be accommodated by any of their nearby edge servers, due to the adequate available resources of those



(a) Time Overhead vs. n (Set #1.1) (b) Time Overhead vs. m (Set #1.2) (c) Time Overhead vs. τ (Set #1.3)

Fig. 7. Time Overhead (Set #1).



(a) Time Overhead vs. n (Set #2.1) (b) Time Overhead vs. m (Set #2.2) (c) Time Overhead vs. τ (Set #2.3)

Fig. 8. Time Overhead (Set #2).

edge servers. In this case, Optimal can find the optimal MUA solution easily and does not need to spend extra time.

Fig. 8 illustrates the average time overhead of all the approaches in Set #2.1 - Set #2.3 across 100 experiment runs. Overall, MUAGame takes the most time to reach a Nash equilibrium, while the other three approaches take almost no time, less than 10 milliseconds in all cases. The time overhead of all five methodologies generally exhibits a linear increase with respect to n and m, as depicted in Fig. 8(a) and (b). The increase in MUAGame's time overhead is the most obvious since more iterations are needed. Let us now take a close look at AirMUA. A detailed examination of AirMUA reveals that its time overhead increases in response to rising n and m are from 0.77 to 4.31 by 28.21%, from 0.36 to 4.89 by 47.19%, respectively, all in milliseconds. As shown in Fig. 8(c), the time overhead of AirMUA decreases when τ increases from 2 to 9. The increase in τ allows individual edge servers to allocate more users around them. As demonstrated earlier in Fig. 6(c) ($\tau \geq 4$), the allocation cost achieved by AirMUA decreases. Thus, Algorithm 2 takes fewer iterations to accommodate all the users, which leads to a decrease in AirMUA's overall time overhead from 33.62 to 1.91 milliseconds when τ increases from 2 to 9, as shown in Fig. 8(c). The results of Set #2 indicate that AirMUA can solve the DMUA problem efficiently. It can tackle the issue of user mobility by rapidly allocating users moving into a new edge server's coverage as new users in the next time slot.

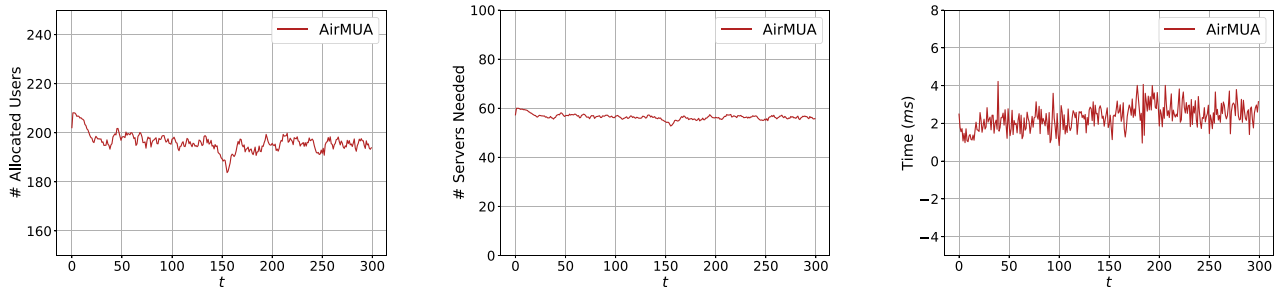
F. Performance Over Time

The average results demonstrated and discussed above indicate AirMUA's performance in general, but not its performance over time. For example, certain cases (if any) where AirMUA fails to allocate most users may be averaged out by the other cases. Its efficiency and stability are also critical. If AirMUA takes a long time to find a solution in any one of the 300 time slots, it will not be able to allocate users in a timely manner in that time slot, which also impacts the allocation of the users in the following time slots. To demonstrate AirMUA's performance over time, Fig. 9 illustrates the experimental results of allocation profit, allocation cost, and time overhead. According to the experimental results, AirMUA's performance fluctuates only slightly over time when users enter and exit the CLSLM system in a random manner. From a statistical standpoint, the standard deviations of its allocation profit, allocation cost, and time overhead in this experiment are 0.57, 0.01, and 0.11, respectively. This shows the high stability of AirMUA over time.

V. Related Work

Recently, mobile edge computing (MEC) has been acknowledged as a promising solution in the 5G era to minimize users' end-to-end latency. Offering many novel opportunities, it has also given rise to a multitude of novel research avenues, encompassing edge application deployment [10], [29], edge machine learning [21], [22], [30], [31], edge computation offloading [13], [32], [33], and mobile user allocation (MUA) [11], [14], [15], [34].

A significant body of research has started to investigate the computation offloading problem with various optimization objectives from the edge infrastructure provider's perspective [4], [6], [20], [35], e.g., maximum energy efficiency, minimum network latency, and maximum system throughput. From the vantage point of application providers, such as YouTube and



(a) Allocation Profit (b) Allocation Cost (c) Overall Time Consumption

Fig. 9. Overall Performance Over Time Slots.

Uber, their users must be served by edge servers. However, edge server resources are typically limited and costly [7], [15], [36]. In addition, users or tasks can only be allocated to nearby edge servers - this is the other unique constraint that differentiates edge computing from cloud computing. These new characteristics raise the new problem of mobile user allocation (MUA) [15], [16], [23].

The MUA problem was first investigated in [23] to maximize the allocation profit first and then minimize the allocation cost. It fundamentally differs from the computation offloading problem in three ways. First, MUA is studied from the application provider's perspective with concerns about costs, while the computation offloading problem focuses on system performance like throughput, latency, etc. Second, users must be allocated to edge servers to access applications or services (or from the cloud if it cannot be allocated to any edge servers) while computation loading allows users to perform tasks on their own devices. Third, MUA considers users' multi-dimensional resource demands while computation offloading focuses on CPU utilization on edge servers. The authors of [16] proposed an efficient heuristic pursuit of allocation profit and allocation cost in an offline manner. The authors of [15] aimed to minimize the system cost. However, existing MUA approaches tried to tackle the MUA problem offline. Their performance is poor when users may come and go dynamically. In such scenarios, users need to be allocated on the fly based on the available resources on edge servers. Recently, Cui et al. [28] proposed an approach to address the MUA problem, focusing on maximizing mobile users' overall data rate under a non-orthogonal multiple access scheme. Lai et al. [34] tried to attack this problem based on Lyapunov optimization. However, similar to many other approaches based on Lyapunov, it suffers from extremely low efficiency and thus contradicts the demanding requirement for low latency in CLSLM systems. To tackle these challenges, in this paper, AirMUA is proposed to tackle the DMUA problem online in the Collaborative Large-Small Language Model environment.

VI. Conclusion and Future Work

In this paper, we tried to solve the novel dynamic mobile user allocation (DMUA) problem in collaborative large-small language model (CLSLM) systems. Initially, we formulate the problem models to provide a foundational framework. Subsequently, we propose AirMUA, an online user allocation approach, to address the DMUA problem effectively. We evaluated its performance comprehensively against several representative approaches. The results demonstrated the high effectiveness, efficiency, and stability of AirMUA. This paper builds the foundation for further studies of the DMUA problem for specific applications, e.g., video streaming, online gaming, and web AR.

In the future, we will integrate AirMUA app-specific models to measure users' quality of experience and resource consumption on edge servers.

References

[1] W. Miao, G. Min, Z. Yu, and X. Zhang, "Performance analytical modelling of mobile edge computing for mobile vehicular applications: A worst-case perspective," *IEEE Trans. Mobile Comput.*, vol. 23, no. 9, pp. 8951–8964, Sep. 2024.

[2] S. Wang, R. Morabito, S. Hosseinalipour, M. Chiang, and C. G. Brinton, "Device sampling and resource optimization for federated learning in cooperative edge networks," *IEEE/ACM Trans. Netw.*, vol. 32, no. 5, pp. 4365–4381, Oct. 2024.

[3] L. T. Hoang, C. T. Nguyen, and A. T. Pham, "Deep reinforcement learning-based online resource management for UAV-assisted edge computing with dual connectivity," *IEEE/ACM Trans. Netw.*, vol. 31, no. 6, pp. 2761–2776, Dec. 2023.

[4] Q. Cai, Y. Zhou, L. Liu, Y. Qi, and J. Shi, "Prioritized assignment with task dependency in collaborative mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13505–13521, Dec. 2024.

[5] C. Huang, G. Chen, P. Xiao, Y. Xiao, Z. Han, and J. A. Chambers, "Joint offloading and resource allocation for hybrid cloud and edge computing in sagins: A decision assisted hybrid action space deep reinforcement learning approach," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 5, pp. 1029–1043, May 2024.

[6] X. Dai, Z. Xiao, H. Jiang, and J. C. S. Lui, "UAV-assisted task offloading in vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2520–2534, Apr. 2024.

[7] Z. Xu et al., "Near-optimal and collaborative service caching in mobile edge clouds," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 4070–4085, Jul. 2023.

[8] D. V. Huynh, V. -D. Nguyen, S. R. Khosravirad, G. K. Karagiannidis, and T. Q. Duong, "Distributed communication and computation resource management for digital twin-aided edge computing with short-packet communications," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 10, pp. 3008–3021, Oct. 2023.

[9] G. Cui et al., "Demand response in NOMA-based mobile edge computing: A two-phase game-theoretical approach," *IEEE Trans. Mobile Comput.*, vol. 22, no. 3, pp. 1449–1463, Mar. 2023.

[10] J. Zhou, Q. Yang, L. Zhao, H. Dai, and F. Xiao, "Mobility-aware computation offloading in satellite edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9135–9149, Oct. 2024.

[11] G. Cui, Q. He, F. Chen, Y. Zhang, H. Jin, and Y. Yang, "Interference-aware game-theoretic device allocation for mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 11, pp. 4001–4012, Nov. 2022.
[12] S. Yue et al., "TODG: Distributed task offloading with delay guarantees for edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 7, pp. 1650–1665, Jul. 2022.
[13] B. Wu, T. Chen, K. Yang, and X. Wang, "Edge-centric bandit learning for task-offloading allocations in multi-RAT heterogeneous networks," *IEEE Trans. Veh. Technol.*, vol. 70, no. 4, pp. 3702–3714, Apr. 2021.
[14] G. Cui, Q. He, F. Chen, H. Jin, and Y. Yang, "Trading off between multi-tenancy and interference: A service user allocation game," *IEEE Trans. Serv. Comput.*, vol. 15, no. 4, pp. 1980–1992, Jul./Aug. 2022.
[15] Q. He et al., "A game-theoretical approach for user allocation in edge computing environment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 3, pp. 515–529, Mar. 2020.
[16] P. Lai et al., "Cost-effective user allocation in 5G NOMA-based mobile edge computing systems," *IEEE Trans. Mobile Comput.*, vol. 21, no. 12, pp. 4263–4278, Dec. 2022.
[17] P. Lai et al., "Quality of experience-aware user allocation in edge computing systems: A potential game," in *Proc. 40th IEEE Int. Conf. Distrib. Comput. Syst.*, 2020, pp. 223–233.
[18] Z. Liu, K. Liu, M. Guo, S. Zhang, and Y. Wang, "Cotuning: A large-small model collaborating distillation framework for better model generalization," in *Proc. 32nd ACM Int. Conf. Multimedia*, 2024, pp. 10487–10496.
[19] P. Lai et al., "Online user and power allocation in dynamic NOMA-based mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 11, pp. 6676–6689, Nov. 2023.
[20] D. Wu, Z. Wang, H. Pan, H. Yao, T. Mai, and S. Guo, "In-network computing empowered mobile edge offloading architecture for Internet of Things," *IEEE Trans. Serv. Comput.*, vol. 17, no. 6, pp. 3817–3829, Nov./Dec. 2024.
[21] R. Li, T. Ouyang, L. Zeng, G. Liao, Z. Zhou, and X. Chen, "Online optimization of DNN inference network utility in collaborative edge computing," *IEEE/ACM Trans. Netw.*, vol. 32, no. 5, pp. 4414–4426, Oct. 2024.
[22] P. Wang et al., "Decentralized navigation with heterogeneous federated reinforcement learning for UAV-enabled mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13621–13638, Dec. 2024.
[23] P. Lai et al., "Optimal edge user allocation in edge computing with variable sized vector bin packing," in *Proc. Int. Conf. Service-Oriented Comput.*, 2018, pp. 230–245.
[24] T. Ouyang, X. Chen, L. Zeng, and Z. Zhou, "Cost-aware dispersed resource probing and offloading at the edge: A user-centric online layered learning approach," *IEEE Trans. Serv. Comput.*, vol. 17, no. 6, pp. 3270–3285, Nov./Dec. 2024.
[25] A. Ghodsi, M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica, "Dominant resource fairness: Fair allocation of multiple resource types," in *Proc. 8th USENIX Conf. Networked Syst. Des. Implementation*, 2011, pp. 323–336.
[26] G. Cui, Q. He, X. Xia, F. Chen, and Y. Yang, "EEsaver: Saving energy dynamically for green multi-access edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 7, pp. 2155–2166, Jul. 2023.
[27] J. Zhou, F. Chen, G. Cui, Y. Xiang, and Q. He, "FEUAgame: Fairness-aware edge user allocation for app vendors," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 8, pp. 1429–1443, Aug. 2024.
[28] G. Cui et al., "OL-EUA: Online user allocation for NOMA-based mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 4, pp. 2295–2306, Apr. 2023.
[29] Y. Qu et al., "Server placement for edge computing: A robust submodular maximization approach," *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 3634–3649, Jun. 2023.
[30] O. Prabhune, T. Chen, and Y. Kim, "Content-aware input scaling and deep learning computation offloading for low-latency embedded vision," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 2218–2226.
[31] Q. Chen et al., "Towards real-time inference offloading with distributed edge computing: The framework and algorithms," *IEEE Trans. Mobile Comput.*, vol. 23, no. 7, pp. 7552–7571, Jul. 2024.
[32] H. Jiang, X. Dai, Z. Xiao, and A. Iyengar, "Joint task offloading and resource allocation for energy-constrained mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 4000–4015, Jul. 2023.
[33] Y. Cheng, C. Liang, Q. Chen, and F. R. Yu, "An efficient resource allocation scheme with uncertain network status in edge computing-enabled networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 3, pp. 1249–1263, Mar. 2025.
[34] P. Lai et al., "Dynamic user allocation in stochastic mobile edge computing systems," *IEEE Trans. Serv. Comput.*, vol. 15, no. 5, pp. 2699–2712, Sep./Oct. 2022.
[35] X. Gong, M. Chen, D. Li, and Y. Cao, "Delay-optimal distributed computation offloading in wireless edge networks," *IEEE/ACM Trans. Netw.*, vol. 32, no. 4, pp. 3376–3391, Aug. 2024.
[36] X. Xia, F. Chen, J. Grundy, M. Abdelrazek, H. Jin, and Q. He, "Constrained app data caching over edge server graphs in edge computing environment," *IEEE Trans. Serv. Comput.*, vol. 15, no. 5, pp. 2635–2647, Sep./Oct. 2022.

Guangming Cui received the master's degree from Anhui University, Hefei, China, in 2018, and the PhD degree in computer science from the Swinburne University of Technology, Melbourne, VIC, Australia, in 2022. He is currently an associate professor with the Nanjing University of Information Science & Technology, China. He has authored or coauthored more than 30 peer-reviewed articles in international journals and conferences, including *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Journal on Selected Areas in Communications*, ICWS, and ICSOC. His research interests include edge computing, service computing, mobile computing, and software engineering.

Xiaolong Xu (Senior Member, IEEE) received the PhD degree in computer science and technology from Nanjing University, Nanjing, China, in 2016. He is currently a full professor with the School of Software, Nanjing University of Information Science and Technology. He has authored or coauthored more than 100 peer-reviewed articles in international journals and conferences, including *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Fuzzy Systems*, *IEEE Transactions on Intelligent Transportation Systems*, IJCAI, ICDM, ICWS, and ICSOC. His research interests include edge computing, Internet of Things, cloud computing, and Big Data. He was selected as the Highly Cited Researcher by Clarivate from 2021 to 2023. He was the recipient of the Best Paper Awards from Tsinghua Science and Technology in 2023, Journal of Network and Computer Applications in 2022, and several conferences, including IEEE HPCC 2023, IEEE ISPA 2022, IEEE CyberSciTech 2021, and IEEE CPSCom2020.

Yiwen Zhang received the PhD degree in management science and engineering from the Hefei University of Technology, in 2013. He is currently a professor with the School of Computer Science and Technology, Anhui University. His research interests include service computing, cloud computing, and Big Data analytics. Please see more information on our website http://bigdata.ahu.edu.cn/.

Lianyong Qi (Senior Member, IEEE) received the PhD degree from the Department of Computer Science and Technology, Nanjing University, China. In 2010, he visited the Department of Information and Communication Technology, Swinburne University of Technology, Australia. He is currently a professor with the College of Computer Science and Technology, China University of Petroleum (East China), China. He has authored or coauthored more than 100 peer-reviewed articles in international journals and conferences, including *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Fuzzy Systems*, *IEEE Transactions on Intelligent Transportation Systems*, IJCAI, ICDM, ICWS, and ICSOC. His research interests include services computing, Big Data, and the Internet of Things.

Zhipeng Cai (Fellow, IEEE) received the BS degree from the Beijing Institute of Technology, Beijing, China, in 2001, and the MS and PhD degrees from the Department of Computing Science, University of Alberta, Edmonton, AB, Canada, in 2004 and 2008, respectively. He is currently a professor with the Department of Computer Science, Georgia State University, Atlanta, GA, USA. His research has received funding from multiple academic and industrial sponsors, including the National Science Foundation and the U.S. Department of State, and has resulted in more than 100 publications in top journals and conferences, with more than 14500 citations, including more than 80 IEEE/ACM transactions papers. His research expertise lies in the areas of resource management and scheduling, privacy, networking, and Big Data.